



VRIJE
UNIVERSITEIT
BRUSSEL



Graduation thesis submitted in partial fulfilment of the requirements for the
degree of de Ingenieurswetenschappen: Computerwetenschappen

PROGRAMMING ROBOTS WITH HIGHER-ORDER REACTIVE PROGRAMMING

G rard Lichtert

2025-2026

Promotor: Prof. Dr. Wolfgang De Meuter
Advisor: Bjarno Oeyen

Sciences and Bio-engineering Sciences



VRIJE
UNIVERSITEIT
BRUSSEL



Proefschrift ingediend met het oog op het behalen van de graad van Master of
Science in de Ingenieurswetenschappen: Computerwetenschappen

ROBOTS PROGRAMMEREN MET HOGERE-ORDE REACTIEF PROGRAMMEREN

Gérard Lichtert

2025-2026

Promotor: Prof. Dr. Wolfgang De Meuter
Advisor: Bjarno Oeyen

Wetenschappen en Bio-ingenieurswetenschappen

Abstract

Reactive programming (RP) is a declarative programming paradigm based on data streams and the propagation of change. In a nutshell, if we have two variables x and y and another variable z that sums both x and y , it should be sufficient to update either x or y and z should be updated automatically.

There are several approaches to achieve reactive programming. One approach is to create a dedicated language. In this case the reactivity will be native to the language. Another option is to extend a language with reactive features, like

Contents

1	Introduction	1
1.1	Contributions	2
1.2	Overview of this Dissertation	2
1.3	Approach	2
2	State of the Art	3
2.1	Reactive Programming	3
2.2	RP languages targeting MCUs	3
2.3	Systems to program robots	3
2.4	Tierless Programming	3
2.5	Problem Statement	3
3	μ-Remus	5
3.1	Haai & Remus	5
3.2	ROS & μ -ROS	5
3.3	Mapping RP to ROS & μ -ROS	5
3.4	μ -Remus: A C implementation	5
4	Programming Robots with Reactors	7
4.1	Linking ROS and μ -ROS	7

5	Tierless Haai	9
5.1	Haai to CHaai	9
6	Evaluation	11
6.1	Evaluating Haai distribution	11
6.2	Evaluating Haai to CHaai	11
6.3	Evaluating CHaai on μ -Remus	11
7	Conclusion & Future Work	13

Chapter 1

Introduction

Modern software is written in such a way that it is event-driven, or in other words, reactive. This means that applications reacts to an incoming event by creating subsequent events that arise as a reaction to the incoming event. Such an application is often called a reactive system. Reactive systems are often implemented with callbacks or by implementing the Observer Pattern. However, this does not come without its drawbacks as this may lead to ‘inversion of control’ or ‘callback hell’[1][2]. To alleviate this we introduce **Reactive Programming**. A programming paradigm that allows the user to write reactive systems in a declarative way. Traditionally the sum and the product of two elements would be written as shown in Listing 1.

```
1 int x = 5;
2 int y = 10;
3 int sum = x + y;
4 int product = x * y;
5 // x changes state
6 x = 10;
7 // sum requires recomputation
8 sum = x + y;
9 // product requires recomputation
10 product = x * y;
11
```

Listing 1: Imperative product and sum

Whenever x or y would change state, you would have to manually update sum and the product. Reactive programming allows you to declare x and y as sources of data and sum , $product$ as the result of summing x and x . sum can in turn be propagated to a sink, which could trigger other events in the system. An example of how such a reactive program is written is shown in Listing ?? There are different ways to achieve Reactive Programming. One way is to create a reactive programming language, making the language itself reactive. Another is to extend a language with reactive features, either by introducing native reactivity or through a library such as ReactiveX. We will be looking at the former way and more specifically into the higher-order reactive programming language **Haai**[3].

```
1 (defr (sum x y)
2   (def result (+ x y))
3   (out result))
4
```

Additionally, we want to delve into the world of electronics. From maintaining your fridge's temperature to controlling a drone's altitude, electronics are deeply embedded in everyone's day-to-day life. Writing software for these electronics can also be a challenging. This mainly due to the fact that using peripherals require using their associated libraries, which are not always standardised. As well as using a low-level programming language such as C/C++ or Rust, which come with their own challenges. Nowadays there are some efforts being made to provide a standardised collection of tools to aid development of robots. One of these tools is called **Robot Operating Systems (ROS)**. What we want to achieve with this thesis is to be able to write a reactive program in Haai for microcontrollers making use of ROS.

1.1 Contributions

1.2 Overview of this Dissertation

1.3 Approach

Chapter 2

State of the Art

2.1 Reactive Programming

2.2 RP languages targeting MCUs

2.3 Systems to program robots

2.4 Tierless Programming

2.5 Problem Statement

Chapter 3

μ -Remus

3.1 Haai & Remus

3.2 ROS & μ -ROS

3.3 Mapping RP to ROS & μ -ROS

3.4 μ -Remus: A C implementation

Chapter 4

Programming Robots with Reactors

4.1 Linking ROS and μ -ROS

Chapter 5

Tierless Haai

5.1 Haai to CHaai

Chapter 6

Evaluation

6.1 Evaluating Haai distribution

6.2 Evaluating Haai to CHaai

6.3 Evaluating CHaai on μ -Remus

Chapter 7

Conclusion & Future Work

Bibliography

- [1] S. Van den Vonder, B. Oeyen, G. Salvaneschi, W. De Meuter and J. Nicolay, ‘A survey on reactive programming and reactive streams,’ *ACM Computing Surveys*, Nov. 2024, Manuscript submitted to ACM.
- [2] E. Bainomugisha, A. Lombide Carreton, T. Van Cutsem, S. Mostinckx and W. De Meuter, ‘A survey on reactive programming,’ *ACM Comput. Surv.*, vol. 45, no. 4, Aug. 2013, issn: 0360-0300. DOI: 10.1145/2501654.2501666 [Online]. Available: <https://doi.org/10.1145/2501654.2501666>
- [3] B. Oeyen, J. De Koster and W. De Meuter, ‘Reactive programming without functions,’ *The Art, Science, and Engineering of Programming*, vol. 8, no. 3, 11:1–11:30, 2024. DOI: 10.22152/programming-journal.org/2024/8/11